フォーマット管理API仕様書

作成日: 2026-01-21 16:35
最終更新: 2026-02-19 15:22
バージョン: 1.1.0
対象画面: 実験管理者 - フォーマット管理画面
総API数: 5個
実装状況: 2個実装済み、3個未実装

📊 実装状況サマリー

カテゴリ	実装済み	未実装	合計
フォーマット管理	2	0	2
ファイル管理	0	2	2
フォルダ管理	0	1	1
合計	**2**	**3**	**5**

完了率: 40% (2/5)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態	優先度
1	フォーマット一覧取得	GET	`/api/v1/formats`	✅ 実装済み	⭐⭐⭐ 高
2	フォーマットファイル一覧取得	GET	`/api/v1/formats/:folderName/files`	✅ 実装済み	⭐⭐⭐ 高
3	新規ファイル作成	POST	`/api/v1/formats/:formatId/files`	🔴 未実装	⭐⭐ 中
4	新規フォルダ作成	POST	`/api/v1/formats/:formatId/folders`	🔴 未実装	⭐⭐ 中

5	ファイル編集（保存）	PUT	`/api/v1/formats/:formatId/files/:fileId`	🔴 未実装	⭐⭐⭐ 高

📋 概要

フォーマット管理APIは、MAIMLワークフローテンプレートと各種XMLテンプレートファイルの管理を提供します。

フォーマットの種類

データベース管理:

- `format` テーブル: フォーマット（プレフィックス）の管理
- `format_subfolder` テーブル: サブフォルダの階層構造管理

ファイルの種類:

1. **ワークフローMAIML（.maiml）**: Petri Net形式の実験手順
2. **テンプレートXML（.xml）**: Material、Condition、Resultテンプレート

S3構造:

```
org_{organization_prefix}/formats/
├── chemistry/                    # format_prefix（例: chemistry）
│   ├── synthesis/               # サブフォルダ（任意の階層）
│   │   └── workflow.maiml
│   └── analysis/
│       └── analysis_workflow.maiml
├── materials/
│   ├── material_template.xml
│   └── conditions/
│       └── condition_template.xml
└── biology/
    └── cell_culture.maiml
```

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、 `Authorization` ヘッダーのセッショントークンから自動的に取得されます
- リクエストヘッダーで `X-Organization-Id` を指定する必要はありません
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

権限要件:

- すべてのAPI: `experiment_admin` または `system_admin` ロール必須

ミドルウェアチェーン:

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  permissionCheck([...]),   // 2. 権限チェック
  controller.handler        // 3. コントローラー処理
);
```

ミドルウェアの動作:

1. `authMiddleware` : JWTトークン（ `maiml_session-{sessionId}_{timestamp}` ）を検証し、セッション情報を取得して `req.user` に格納（organizationIdを含む）
2. `permissionCheck` : ユーザーのロール（ `req.user.role` ）を検証

3. コントローラー: `req.user.organizationId` を使用してテナント分離されたデータにアクセス

マルチテナント分離

- すべてのDBクエリに `organization_id` フィルタ必須
- すべてのS3パスに `org_{organization_prefix}` プレフィックス必須

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

📋 API詳細仕様

1. フォーマット一覧取得 🔴

エンドポイント: `GET /api/v1/formats`

説明: 組織内のフォーマット（プレフィックス）一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`file_type`	string	×	all	ファイル種別フィルタ（`all`, `maiml`, `xml`）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": [
    {
      "format_id": "format-uuid",
      "format_prefix": "chemistry",
      "file_type": "maiml",
      "description": "化学実験用ワークフロー",
      "subfolder_count": 3,
      "file_count": 8,
      "created_at": "2026-01-21T10:00:00Z",
      "updated_at": "2026-01-21T16:00:00Z"
    },
    {
      "format_id": "format-uuid-2",
      "format_prefix": "materials",
      "file_type": "xml",
```

```
      "description": "材料テンプレート",
      "subfolder_count": 2,
      "file_count": 5,
      "created_at": "2026-01-20T10:00:00Z",
      "updated_at": "2026-01-21T15:00:00Z"
    }
  ],
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 401: 認証エラー
- 403: 権限不足
- 400: バリデーションエラー
- 500: システムエラー

実装要件

DBクエリ:

```
// formatテーブルから取得
const formats = await db
  .selectFrom('format')
  .where('organization_id', '=', organizationId)
  .leftJoin('format_subfolder', 'format.format_id', 'format_subfolder.format_id')
  .select([
    'format.format_id',
    'format.format_prefix',
    'format.file_type',
    'format.description',
    db.fn.countAll<number>().as('subfolder_count')
  ])
  .groupBy(['format.format_id', 'format.format_prefix', 'format.file_type',
'format.description'])
  .execute();

// S3からファイル数を取得
for (const format of formats) {
  const files = await storageService.listFiles(
    organizationPrefix,
```

```
    `formats/${format.format_prefix}`
  );
  format.file_count = files.length;
}
```

---

2. フォーマットファイル一覧取得 🔴

エンドポイント: `GET /api/v1/formats/:formatId/files`

説明: 指定フォーマットのファイル・フォルダー一覧を階層構造で取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`formatId`	UUID	○	フォーマットID

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`subfolder_id`	UUID	×	null	サブフォルダID（指定時はそのフォルダ配下のみ）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "format_id": "format-uuid",
    "format_prefix": "chemistry",
    "file_type": "maiml",
    "current_path": "chemistry/synthesis",
    "folders": [
```

```
      {
        "subfolder_id": "subfolder-uuid",
        "subfolder_name": "synthesis",
        "subfolder_path": "chemistry/synthesis",
        "file_count": 3,
        "created_at": "2026-01-21T10:00:00Z"
      },
      {
        "subfolder_id": "subfolder-uuid-2",
        "subfolder_name": "analysis",
        "subfolder_path": "chemistry/analysis",
        "file_count": 2,
        "created_at": "2026-01-21T11:00:00Z"
      }
    ],
    "files": [
      {
        "file_id": "file-uuid",
        "file_name": "workflow.maiml",
        "file_path": "org_maiml_lab_001/formats/chemistry/workflow.maiml",
        "file_size": 12345,
        "file_type": "maiml",
        "created_at": "2026-01-21T10:00:00Z",
        "updated_at": "2026-01-21T16:00:00Z"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: フォーマットが存在しない
- 500: システムエラー

実装要件

処理フロー:

1. formatテーブルからformat_prefix取得
2. subfolder_idが指定されている場合、そのフォルダパスを取得
3. format_subfolderテーブルから直下のサブフォルダ一覧取得
4. S3から指定パス配下のファイル一覧取得
5. file_metadataテーブルと照合してメタデータ付与

S3パス構造:

```
const basePath = subfolderId
  ? `formats/${format_prefix}/${subfolder_path}`
  : `formats/${format_prefix}`;

const files = await storageService.listFiles(
  organizationPrefix,
  basePath
);
```

---

3. 新規ファイル作成 🔴

エンドポイント: `POST /api/v1/formats/:formatId/files`

説明: 新規ファイルを作成（空のMAIMLまたはXMLテンプレート）

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`formatId`	UUID	○	フォーマットID

ボディ（JSON）:

```
{
  "fileName": "new_workflow.maiml",
  "subfolderId": "subfolder-uuid",
  "templateType": "empty"
}
```

バリデーション:

フィールド	型	必須	制約
`fileName`	string	○	1-255文字、拡張子必須（`.maiml` または `.xml`）
`subfolderId`	UUID	×	有効なサブフォルダID
`templateType`	string	×	`empty`, `basic`, `advanced`

レスポンス

成功時 (201 Created):

```
{
  "success": true,
  "data": {
    "file_id": "file-uuid",
    "file_name": "new_workflow.maiml",
    "file_path": "org_maiml_lab_001/formats/chemistry/synthesis/new_workflow.maiml",
    "file_size": 1234,
    "file_type": "maiml",
    "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
    "created_at": "2026-01-21T16:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、拡張子不正
- 401: 認証エラー
- 403: 権限不足

- 404: フォーマットまたはサブフォルダが存在しない
- 409: ファイル名重複
- 500: システムエラー

実装要件

テンプレート生成:

```
// MAIMLの場合
const maimlTemplate = `<?xml version="1.0" encoding="UTF-8"?>
<maiml>
  <project>
    <projectName>${fileName.replace('.maiml', '')}</projectName>
  </project>
  <methods>
    <method id="method1" name="Method 1">
      <petriNet>
        <!-- Empty Petri Net -->
      </petriNet>
    </method>
  </methods>
</maiml>`;

// XMLテンプレートの場合
const xmlTemplate = `<?xml version="1.0" encoding="UTF-8"?>
<template type="${templateType}">
  <!-- Template content -->
</template>`;
```

S3保存:

```
const s3Key = subfolderId
  ? `formats/${format_prefix}/${subfolder_path}/${fileName}`
  : `formats/${format_prefix}/${fileName}`;

await storageService.uploadFile(
  organizationPrefix,
  s3Key,
  Buffer.from(template, 'utf-8'),
  { contentType: 'application/xml' }
);
```

---

4. 新規フォルダ作成 🔴

エンドポイント: `POST /api/v1/formats/:formatId/folders`

説明: 新規サブフォルダを作成

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`formatId`	UUID	○	フォーマットID

ボディ（JSON）:

```
{
  "subfolderName": "new_category",
  "parentSubfolderId": "parent-subfolder-uuid",
  "description": "新しいカテゴリ"
}
```

バリデーション:

フィールド	型	必須	制約
`subfolderName`	string	○	1-255文字、英数字・アンダースコアのみ
`parentSubfolderId`	UUID	×	親フォルダID（未指定時はルート直下）
`description`	string	×	0-500文字

レスポンス

成功時 (201 Created):

```
{
  "success": true,
  "data": {
    "subfolder_id": "subfolder-uuid",
    "format_id": "format-uuid",
    "subfolder_name": "new_category",
    "subfolder_path": "chemistry/synthesis/new_category",
    "parent_subfolder_id": "parent-uuid",
    "description": "新しいカテゴリ",
    "created_at": "2026-01-21T16:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、フォルダ名不正
- 401: 認証エラー
- 403: 権限不足
- 404: フォーマットまたは親フォルダが存在しない
- 409: フォルダ名重複
- 500: システムエラー

実装要件

DBトランザクション:

```
await db.transaction().execute(async (trx) => {
  // 親フォルダのパスを取得
  const parentPath = parentSubfolderId
    ? await trx.selectFrom('format_subfolder')
        .where('subfolder_id', '=', parentSubfolderId)
        .select('subfolder_path')
        .executeTakeFirstOrThrow()
    : null;

  // 新しいパスを生成
  const subfolderPath = parentPath
    ? `${parentPath.subfolder_path}/${subfolderName}`
```

```
    : `${format_prefix}/${subfolderName}`;

  // format_subfolderに登録
  const subfolder = await trx.insertInto('format_subfolder')
    .values({
      subfolder_id: uuidv4(),
      format_id: formatId,
      subfolder_name: subfolderName,
      subfolder_path: subfolderPath,
      parent_subfolder_id: parentSubfolderId,
      description,
      organization_id: organizationId,
      created_at: new Date(),
      updated_at: new Date()
    })
    .returningAll()
    .executeTakeFirstOrThrow();

  return subfolder;
});
```

S3フォルダ作成:

- S3にはフォルダという概念がないため、DBのみで管理
- ファイルアップロード時にパスとして使用

---

5. ファイル編集（保存） 🔴

エンドポイント: `PUT /api/v1/formats/:formatId/files/:fileId`

説明: ファイル内容を保存（上書き）

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明

`formatId`	UUID	○	フォーマットID
`fileId`	UUID	○	ファイルID

ボディ（JSON）:

```
{
  "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
  "createBackup": true
}
```

バリデーション:

フィールド	型	必須	制約
`content`	string	○	XML形式、最大10MB
`createBackup`	boolean	×	バックアップ作成フラグ（デフォルト: false）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "file_id": "file-uuid",
    "file_name": "workflow.maiml",
    "file_path": "org_maiml_lab_001/formats/chemistry/workflow.maiml",
    "file_size": 15678,
    "updated_at": "2026-01-21T16:00:00Z",
    "backup_created": true,
    "backup_path": "org_maiml_lab_001/formats/chemistry/.backup/workflow_20260121160000.maiml"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、XML形式不正、ファイルサイズ超過

- 401: 認証エラー
- 403: 権限不足
- 404: フォーマットまたはファイルが存在しない
- 500: システムエラー

実装要件

バックアップ処理:

```
if (createBackup) {
  // 既存ファイルをバックアップフォルダにコピー
  const timestamp = new Date().toISOString().replace(/[:.]/g, '').substring(0, 14);
  const backupPath =
`formats/${format_prefix}/.backup/${fileNameWithoutExt}_${timestamp}.${ext}`;

  const existingContent = await storageService.downloadFile(
    organizationPrefix,
    originalFilePath
  );

  await storageService.uploadFile(
    organizationPrefix,
    backupPath,
    existingContent,
    { contentType: 'application/xml' }
  );
}
```

XML バリデーション:

```
import { parseString } from 'xml2js';

// XMLパース確認
try {
  await new Promise((resolve, reject) => {
    parseString(content, (err, result) => {
      if (err) reject(new ValidationError('XML形式が不正です'));
      resolve(result);
    });
  });
} catch (error) {
```

```
  throw new ValidationError('XMLパースに失敗しました');
}
```

S3上書き保存:

```
await storageService.uploadFile(
  organizationPrefix,
  filePath,
  Buffer.from(content, 'utf-8'),
  { contentType: 'application/xml' }
);

// file_metadataテーブルを更新
await db.updateTable('file_metadata')
  .set({
    file_size: Buffer.byteLength(content, 'utf-8'),
    updated_at: new Date()
  })
  .where('file_id', '=', fileId)
  .where('organization_id', '=', organizationId)
  .execute();
```

🔒 セキュリティ考慮事項

マルチテナント分離

必須チェック:

- ☐ すべてのDBクエリに `organization_id` フィルタ
- ☐ すべてのS3操作に `org_{organization_prefix}` プレフィックス
- ☐ クロステナントアクセスの防止

実装例:

```
// ✅ 良い例
const format = await db
  .selectFrom('format')
  .where('format_id', '=', formatId)
```

```
  .where('organization_id', '=', organizationId)  // 必須
  .selectAll()
  .executeTakeFirst();

// ❌ 悪い例
const format = await db
  .selectFrom('format')
  .where('format_id', '=', formatId)  // organization_idフィルタなし
  .selectAll()
  .executeTakeFirst();
```

ファイル操作

セキュリティチェック:

- [ ] XMLバリデーション（XXE攻撃対策）
- [ ] ファイルサイズ制限（最大10MB）
- [ ] パストラバーサル攻撃の防止
- [ ] format_prefixの検証

実装例:

```
// XMLパーサーのセキュリティ設定
const parserOptions = {
  explicitArray: false,
  ignoreAttrs: false,
  xmlns: false,
  // XXE攻撃対策
  explicitChildren: false,
  preserveChildrenOrder: false,
  // 外部エンティティを無効化
  strict: true
};
```

📚 関連ドキュメント

規約

- API設計ポリシー
- バックエンド規約
- セキュリティ規約

設計書

- API一覧（統合版）
- API実装方針
- S3構造
- 共通API
- プロジェクト管理API

UI仕様書

- フォーマット管理画面

📝 変更履歴

バージョン	日付	変更内容	担当者
1.1.0	2026-02-19	API #1, #2実装完了（40%達成）	Cline
		- フォーマット一覧取得API実装	
		- フォーマットファイル一覧取得API実装	
		- パスパラメータを `:folderName` に変更（API #2）	
		- サブフォルダ対応（pathクエリパラメータ）	
		- 実装ファイル: formatController.ts, formatService.ts	
1.0.0	2026-01-21	新規作成	Cline

作成者: Cline
作成日: 2026-01-21 16:35
最終更新: 2026-02-19 15:22